

# Lab 11: Programs as Data, Macros

**lab11.zip (lab11.zip)**

---

*Due by 11:59pm on Wednesday, November 20.*

## Starter Files

Download [lab11.zip](#) (lab11.zip).

## Required Questions

---

Getting Started Videos

## Quasiquotation

Consult the drop-down if you need a refresher on quasiquotation. It's okay to skip directly to the questions and refer back here should you get stuck.

Quasiquotation

The normal quote `'` and the quasiquote ``` are both valid ways to quote an expression. However, the quasiquoted expression can be *unquoted* with the "unquote" , (represented by a comma). When a term in a quasiquoted expression is *unquoted*, the unquoted term is *evaluated*, instead of being taken as literal text. This mechanism is somewhat akin to using *f-strings* in Python, where expressions inside `{}` are evaluated and inserted into the string.

```
scm> (define a 5)
a
scm> (define b 3)
b
scm> `(* a b) ; Quasiquoted expression
(* a b)
scm> `(* a ,b) ; Unquoted b, which evaluates to 3
(* a 3)
scm> `(* ,(+ a b) b) ; Unquoted (+ a b), which evaluates to 8
(* 8 b)
```

## Q1: WWSD: Quasiquote

Use Ok to test your knowledge with the following "What Would Scheme Display?" questions:

```
python3 ok -q wwsd-quasiquote -u
```

```
scm> '(1 x 3)

scm> (define x 2)

scm> `(1 x 3)

scm> `(1 ,x 3)

scm> `(1 x ,3)

scm> `(1 (,x) 3)

scm> `(1 ,(+ x 2) 3)

scm> (define y 3)

scm> `(x ,(* y x) y)

scm> `(1 ,(cons x (list y 4)) 5)
```

# Programs as Data

Consult the drop-down if you need a refresher on Programs as Data. It's okay to skip directly to the questions and refer back here should you get stuck.

## Programs as Data

All Scheme programs are made up of expressions. There are two types of expressions: *primitive* (a.k.a *atomic*) expressions and *combinations*. Here are some examples of each:

- *Primitive/atomic* expression: `#f`, `1.7`, `+`
- *Combinations*: `(factorial 10)`, `(/ 8 3)`, `(not #f)`

Scheme represents combinations as a Scheme list. Therefore, a combination can be constructed through list manipulation.

For example, the expression `(list '+ 2 2)` evaluates to the list `(+ 2 2)`, which is also an expression. If we then call `eval` on this list, it will evaluate to `4`. The `eval` procedure takes in one argument `expr` and evaluates `expr` in the current environment.

```
scm> (define expr (list '+ 2 2))
expr
scm> expr
(+ 2 2)
scm> (eval expr)
4
```

Additionally, *quasiquote* is very helpful for building procedures that create expressions. Take a look at the following `add-program`:

```
scm> (define (add-program x y)
...>   `(+ ,x ,y))
add-program
scm> (add-program 3 6)
(+ 3 6)
```

`add-program` takes in two inputs `x` and `y` and returns an expression that, if evaluated, evaluates to the result of adding `x` and `y` together. Within `add-program`, we use a `quasiquote` to build the addition expression `(+ ...)`, and we `unquote` `x` and `y` to get their evaluated values in the addition expression.

## Q2: If Program

In Scheme, the `if` special form allows us to evaluate one of two expressions based on a predicate. Write a program `if-program` that takes in the following parameters:

1. `predicate` : a quoted expression which will evaluate to the condition in our `if` - expression

2. `if-true` : a quoted expression which will evaluate to the value we return if predicate evaluates to true (`#t`)
3. `if-false` : a quoted expression which will evaluate to the value we return if predicate evaluates to false (`#f`)

The program returns a Scheme list that represents an `if` expression in the form: `(if <predicate> <if-true> <if-false>)`. Note that we don't want to evaluate the expression (in our program at least).

Here are some doctests to show this:

```
scm> (define x 1)
scm> (if-program '(= 0 0) '(+ x 1) 'x)
(if (= 0 0) (+ x 1) x)
scm> (eval (if-program '(= 0 0) '(+ x 1) 'x))
2
scm> (if-program '(= 1 0) '(print 3) '(print 5))
(if (= 1 0) (print 3) (print 5))
scm> (eval (if-program '(= 1 0) '(print 3) '(print 5)))
5
```

```
(define (if-program condition if-true if-false)
  'YOUR-CODE-HERE
)
```

Use Ok to test your code:

```
python3 ok -q if-program
```



## Q3: Exponential Powers

Implement a procedure `(pow-expr base exp)` that returns an expression that, when evaluated, raises the number `base` to the power of the nonnegative integer `exp`. The body of `pow-expr` should not perform any multiplication (or exponentiation). Instead, it should just construct an expression containing only the symbols `square` and `*` as well as the number `base` and parentheses. The length of this expression should grow logarithmically with respect to `exp`, rather than linearly.

Examples:

```
scm> (pow-expr 2 0)
1
scm> (pow-expr 2 1)
(* 2 1)
scm> (pow-expr 2 5)
(* 2 (square (square (* 2 1))))
scm> (pow-expr 2 15)
(* 2 (square (* 2 (square (* 2 (square (* 2 1)))))))
scm> (pow-expr 2 16)
(square (square (square (square (* 2 1))))))
scm> (eval (pow-expr 2 16))
65536
```

*Hint:*

1.  $x^{2y} = (x^y)^2$
2.  $x^{2y+1} = x(x^y)^2$

For example,  $2^{16} = (2^8)^2$  and  $2^{17} = 2 * (2^8)^2$ .

You may use the built-in predicates `even?` and `odd?`. Also, the `square` procedure is defined for you.

Here's the [solution to a similar homework problem \(https://cs61a.org/hw/sol-hw07/#q1-pow\)](https://cs61a.org/hw/sol-hw07/#q1-pow).

```
(define (square n) (* n n))

(define (pow-expr base exp)
  'YOUR-CODE-HERE
)
```

Use Ok to test your code:

```
python3 ok -q pow
```



## Macros

A macro is a code transformation that is created using `define-macro` and applied using a call expression. A macro call is evaluated by:

1. Binding the formal parameters of the macro to the **unevaluated** operand expressions of the macro call.
2. Evaluating the body of the macro, which returns an expression.
3. Evaluating the expression returned by the macro in the frame of the original macro call.

```
scm> (define-macro (twice expr) (list 'begin expr expr))
twice
scm> (twice (+ 2 2)) ; evaluates (begin (+ 2 2) (+ 2 2))
4
scm> (twice (print (+ 2 2))) ; evaluates (begin (print (+ 2 2)) (print (+ 2 2)))
4
4
```

## Q4: Repeat

Define `repeat`, a macro that takes a number `n` and an expression `expr`. Calling `repeat` evaluates `expr` in a local frame `n` times, and its value is the final result. You will find the helper function `repeated-call` useful, which takes a number `n` and a zero-argument procedure `f` and calls `f` `n` number of times.

For example, `(repeat (+ 2 3) (print 1))` is equivalent to:

```
(repeated-call (+ 2 3) (lambda () (print 1)))
```

and should evaluate `(print 1)` repeatedly 5 times.

The following expression should print `four` four times:

```
(repeat 2 (repeat 2 (print 'four)))
```

```
(define-macro (repeat n expr)
  `(repeated-call ,n ____))

; Call zero-argument procedure f n times and return the final result.
(define (repeated-call n f)
  (if (= n 1) ____ (begin ____ ____)))
```

Use Ok to test your code:

```
python3 ok -q repeat-lambda
```



Hint: repeat

The `repeated-call` procedure takes a zero-argument procedure, so `(lambda () ____)` must appear in the blank. The body of the lambda is `expr`, which must be unquoted.

Hint: repeated-call

Call `f` on no arguments with `(f)`. If `n` is 1, just call `f`. If `n` is greater than 1, first call `f` and then call `(repeated-call (- n 1) f)`.

# Check Your Score Locally

You can locally check your score on each question of this assignment by running

```
python3 ok --score
```

**This does NOT submit the assignment!** When you are satisfied with your score, submit the assignment to Gradescope to receive credit for it.

# Submit Assignment

---

If you are in a regular section of CS 61A, fill out this lab attendance and feedback form (<https://forms.gle/dHxj8gttNWRy6Ptm9>). (If you are in the mega section, you don't need to fill out the form.)

Then, submit this assignment by uploading any files you've edited **to the appropriate Gradescope assignment**. Lab 00 ([../lab00/#submit-with-gradescope](https://gradescope.com/submissions/61a-s21/lab00/#submit-with-gradescope)) has detailed instructions.